

FINÁLNÍ PROJEKT
č. 2

engeto

Autor: Jaroslav Pour
Datum:

OBSAH

ZADÁNÍ	3
TESTOVACÍ SCÉNÁŘE	4
EXEKUCE TESTŮ	13
BUG REPORT	17

ZADÁNÍ

Cílem finálního projektu je otestovat funkčnost aplikace, která slouží k manipulaci s daty o studentech. Aplikace má rozhraní REST-API, které umožňuje vytvoření, smazání a získání dat..

Přístupové údaje:

Databáze	Default scheme: students Host: test-app.engeto.cz Port: 5433
REST-API	https://test-app.engeto.cz/students
REST-API dokumentace	https://test-app.engeto.cz/docs

Poznámky:

Nezapomeňte, že v IT se data musí někde uložit a poté získat. Proto ověřte, že data jsou správně uložena a získávána z databáze.

Nezapomeňte do testovacích scénářů uvést testovací data, očekávaný výsledek včetně těla odpovědi a stavových kódů.

:

TESTOVACÍ SCÉNÁŘE

Na základě uvedených testovacích scénářů jsem ověřil(a) funkčnost aplikace.

Získání Bearer tokenu

TC-00 – Získání Bearer tokenu

Popis: Ověření získání Bearer tokenu pro autorizované požadavky.

API: POST /login

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /login.
3. Do záložky Body vybrat raw → JSON.
4. Zadat platné přihlašovací údaje.
5. Odeslat požadavek.
6. Ověřit odpověď API.
7. Zkopírovat Bearer token.
8. Použít Bearer token v hlavičce Authorization u všech následujících testovacích scénářů.

Testovací data:

```
{  
  "login": "testing_academy_2026_03_26_3",  
  "password": "WPrEmzjP9p1rDzCS"  
}
```

Očekávaný výsledek:

- HTTP 200 OK
- Bearer token je vrácen v odpovědi.
- Token lze použít pro autorizované požadavky.

POST /students

TC-01 – Vytvoření nového studenta

Popis: Ověření vytvoření nového studenta s platnými údaji.

API: POST /students

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /students.
3. Přidat Bearer token získaný v TC-00.
4. Do těla požadavku vložit platný JSON payload.
5. Odeslat požadavek.
6. Ověřit odpověď API.

7. Ověřit vytvoření záznamu v databázi pomocí SQL dotazu.

Testovací data:

JSON

```
{
  "firstName": "Jan",
  "lastName": "TESTOVACI",
  "email": "jan.testovaci.final2026@example.com",
  "isEuCitizen": true,
  "numberOfFailedStudies": 0
}
```

SQL ověření:

```
SELECT *
FROM students
WHERE email = 'jan.testovaci.final2026@example.com';
```

Očekávaný výsledek:

- HTTP 201 Created
- Student je vytvořen.
- Student je uložen v databázi.

Poznámka: ID vytvořeného studenta bude použito v navazujících testovacích scénářích (GET, PUT a DELETE).

TC-02 – Vytvoření studenta s duplicitním e-mailem

Popis: Ověření, že nelze vytvořit studenta se stejným e-mailem.

API: POST /students

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /students.
3. Přidat Bearer token získaný v TC-00.
4. Do těla požadavku vložit stejná data jako v TC-01.
5. Odeslat požadavek.
6. Ověřit odpověď API.

Testovací data: Stejná jako v TC-01.

Očekávaný výsledek:

- HTTP 400 Bad Request
- Student nebude vytvořen.

TC-03 – Vytvoření studenta bez Bearer tokenu

Popis: Ověření autorizace při chybějícím Bearer tokenu.

API: POST /students

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /students.
3. Nepřidat Bearer token.
4. Do těla požadavku vložit platný JSON payload.
5. Odeslat požadavek.

6. Ověřit odpověď API.

Testovací data: Stejná jako v TC-01.

Očekávaný výsledek:

- HTTP 401 Unauthorized
- Přístup bude zamítnut.

TC-04 – Vytvoření studenta s prázdným payloadem

Popis: Ověření validace vstupních dat.

API: POST /students

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /students.
3. Přidat Bearer token získaný v TC-00.
4. Odeslat prázdný JSON payload {}.
5. Odeslat požadavek.
6. Ověřit odpověď API.

Testovací data:

JSON

```
{  
  
}
```

Očekávaný výsledek:

- HTTP 201 Created
- Student bude vytvořen s výchozími hodnotami podle implementace API.

GET /students/

TC-05 – Získání existujícího studenta

Popis: Ověření získání existujícího studenta podle ID.

API: GET /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit GET požadavek na endpoint /students/{student_id}.
3. Přidat Bearer token získaný v TC-00.
4. Zadat ID studenta vytvořeného v TC-01 (ID získané z odpovědi POST požadavku).
5. Odeslat požadavek.
6. Ověřit odpověď API.
7. Ověřit data v databázi pomocí SQL.

SQL ověření:

```
SELECT *  
FROM students  
WHERE email = 'jan.testovaci.final2026@example.com';
```

Očekávaný výsledek:

- HTTP 200 OK
- API vrátí údaje studenta.
- Vracená data odpovídají databázi.

TC-06 – Získání neexistujícího studenta

Popis: Ověření chování při zadání neexistujícího ID.

API: GET /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit GET požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat neexistující ID (např. 999999).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Očekávaný výsledek:

- HTTP 404 Not Found
- Student nebude nalezen.

TC-07 – Získání studenta bez Bearer tokenu

Popis: Ověření autorizace.

API: GET /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit GET požadavek.
3. Nepřidat Bearer token.
4. Zadat ID studenta vytvořeného v TC-01 (ID získané z odpovědi POST požadavku).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Očekávaný výsledek:

-HTTP 401 Unauthorized

TC-08 – Získání studenta s neplatným ID

Popis: Ověření validace parametru student_id.

API: GET /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit GET požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat neplatné ID (např. abc).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Očekávaný výsledek:

- HTTP 422 Unprocessable Entity
- API vrátí validační chybu.

PUT /students**TC-09 – Úspěšná aktualizace studenta**

Popis: Ověření aktualizace existujícího studenta pomocí platných údajů.

API: PUT /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit PUT požadavek na endpoint /students/{student_id}.
3. Přidat Bearer token získaný v TC-00.
4. Zadat ID studenta vytvořeného v TC-01 (ID získané z odpovědi POST požadavku).
5. Upravit některé údaje (např. FirstName).
6. Odeslat požadavek.
7. Ověřit odpověď API.

Testovací data:

JSON

```
{
  "firstName": "JanUpdated",
  "lastName": "TESTOVACI",
  "email": "jan.testovaci.final2026@example.com",
  "isEuCitizen": true,
  "numberOfFailedStudies": 0
}
```

SQL ověření:

```
SELECT *
FROM students
WHERE email = 'jan.testovaci.final2026@example.com';
```

Očekávaný výsledek:

- HTTP 200 OK

- Student je aktualizován.
- Změny jsou uloženy v databázi.

TC-10 – Aktualizace neexistujícího studenta

Popis: Ověření aktualizace studenta s neexistujícím ID.

API: PUT /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit PUT požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat neexistující ID (např. 9999).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Očekávaný výsledek:

- HTTP 404 Not Found
- Student nebude aktualizován.

TC-11 – Aktualizace bez Bearer tokenu

Popis: Ověření autorizace při chybějícím Bearer tokenu.

API: PUT /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit PUT požadavek.
3. Nepřidat Bearer token.
4. Zadat ID studenta vytvořeného v TC-01 (ID získané z odpovědi POST požadavku).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Očekávaný výsledek:

- HTTP 401 Unauthorized

TC-12 – Aktualizace s neplatnými daty

Popis: Ověření validace při odeslání neplatného payloadu.

API: PUT /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit PUT požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat ID studenta vytvořeného v TC-01 (ID získané z odpovědi POST požadavku).
5. Odeslat prázdný payload {}.
6. Ověřit odpověď API.

Testovací data:

JSON

```
{  
}
```

Očekávaný výsledek:

- HTTP 422 Unprocessable Entity
- Student nebude aktualizován.

DELETE /students

TC-13 – Smazání existujícího studenta

Popis: Ověření smazání existujícího studenta.

API: DELETE /students/{student_id}

Předpoklad: Před spuštěním testu byl vytvořen testovací student pomocí POST /students s e-mailem "delete.test.final2026@example.com". Z odpovědi API bylo získáno jeho ID, které je použito v tomto scénáři. Existence studenta byla ověřena SQL dotazem.

Příprava testovacích dat:

1. Vytvořit nového studenta pomocí endpointu POST /students.
"firstName": "Delete",
"lastName": "Test",
"email": "delete.test.final2026@example.com",
"isEuCitizen": true,
"numberOfFailedStudies": 0
2. Ověřit vytvoření studenta pomocí SQL dotazu v DBeaveru.
3. Z odpovědi POST požadavku získat hodnotu student_id.
4. Toto ID použít v následujícím scénáři DELETE.

Postup:

1. Otevřít Postman.
2. Vytvořit DELETE požadavek na endpoint /students/{student_id}.
3. Přidat Bearer token získaný v TC-00.
4. Do endpointu /students/{student_id} zadat ID získané při přípravě testovacích dat.
5. Odeslat požadavek.
6. Ověřit odpověď API.
7. Ověřit pomocí SQL, zda byl student odstraněn z databáze.

Testovací data:

Student ID: Hodnota student_id získaná z odpovědi POST požadavku při přípravě testovacích dat.

SQL ověření:

```
SELECT *  
FROM students  
WHERE email = 'delete.test.final2026@example.com';
```

Očekávaný výsledek:

- HTTP 200 OK
- Student je odstraněn.
- Student se již nenachází v databázi.

TC-14 – Smazání neexistujícího studenta

Popis: Ověření, že nelze smazat neexistujícího studenta.

API: DELETE /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit DELETE požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat neexistující ID studenta (např. 9999).
5. Odeslat požadavek.
6. Ověřit odpověď API.

Testovací data:

Student ID: 9999

Očekávaný výsledek:

- HTTP 404 Not Found
- Student nebude odstraněn.

TC-15 – Smazání studenta bez Bearer tokenu

Popis: Ověření autorizace při chybějícím Bearer tokenu.

Předpoklad: Existuje testovací student vytvořený v rámci přípravy testovacích dat. Jeho ID bylo získáno z odpovědi POST požadavku.

API: DELETE /students/{student_id}

Příprava testovacích dat:

1. Vytvořit testovacího studenta pomocí endpointu POST /students.
2. Ověřit vytvoření studenta pomocí SQL dotazu v DBeaveru.
3. Z odpovědi POST požadavku získat hodnotu student_id.
4. Toto ID použít v následujícím scénáři DELETE bez Bearer tokenu.

Postup:

1. Otevřít Postman.
2. Vytvořit DELETE požadavek.
3. Nepřidat Bearer token.
4. Zadat ID existujícího testovacího studenta.
5. Odeslat požadavek.
6. Ověřit odpověď API.

Testovací data:

Student ID: ID existujícího testovacího studenta získané z odpovědi POST požadavku.

Očekávaný výsledek:

- HTTP 401 Unauthorized

TC-16 – Smazání studenta s neplatným ID

Popis: Ověření validace při zadání neplatného ID studenta.

API: DELETE /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit DELETE požadavek.
3. Přidat Bearer token získaný v TC-00.
4. Zadat místo ID hodnotu abc.
5. Odeslat požadavek.
6. Ověřit odpověď API.

Testovací data:

Student ID: abc

Očekávaný výsledek:

- HTTP 422 Unprocessable Entity
- Student nebude odstraněn.

TC-17 – Vytvoření a následné získání studenta

Popis: Ověření, že nově vytvořený student je možné následně získat pomocí GET podle ID vráceného při vytvoření.

API: POST /students → GET /students/{student_id}

Postup:

1. Otevřít Postman.
2. Vytvořit POST požadavek na endpoint /students.
3. Přidat Bearer token získaný v TC-00.
4. Do těla požadavku vložit platný JSON payload.
5. Odeslat požadavek.
6. Ověřit odpověď API.
7. Z odpovědi POST požadavku získat hodnotu student_id.
8. Vytvořit GET požadavek na endpoint /students/{student_id}.
9. Do endpointu zadat student_id získané z odpovědi POST požadavku.
10. Přidat Bearer token získaný v TC-00.
11. Odeslat požadavek.
12. Ověřit, že vrácená data odpovídají údajům odeslaným při vytvoření studenta.

Testovací data:

```
JSON {  
    "firstName": "Petr",  
    "lastName": "Novak",  
    "email": "petr.novak.e2e.test@example.com",  
    "isEuCitizen": true,  
    "numberOfFailedStudies": 0  
}
```

Očekávaný výsledek:

- POST vrátí HTTP 201 Created.
- Student je vytvořen.
- Z odpovědi POST je získáno student_id.
- GET vrátí HTTP 200 OK.
- Vrácená data odpovídají údajům použitým při vytvoření studenta.

EXEKUCE TESTŮ

Testovací scénáře jsem provedl(a), přikládám výsledky testů.

TC-00

Výsledek: PASS

API vrátilo HTTP 200 OK.

Bearer token byl úspěšně získán a použit v následujících testech.

TC-01

Výsledek: PASS

API vrátilo HTTP 201 Created.

Student byl úspěšně vytvořen.

Záznam byl ověřen v databázi (ID = 277)

TC-02

Výsledek: PASS

API vrátilo HTTP 400 Bad Request.

Student nebyl vytvořen.

TC-03

Výsledek: FAIL

Očekáváno: HTTP 401 Unauthorized.

Skutečnost: HTTP 403 Forbidden.

TC-04

Výsledek: PASS

Očekáváno: HTTP 201 Created.

Skutečnost: HTTP 201 Created.

Student byl úspěšně vytvořen i při prázdném payloadu. Test byl vyhodnocen jako PASS.

TC-05

Výsledek: PASS

Očekáváno: HTTP 200 OK.

Skutečnost: HTTP 200 OK.

API vrátilo správná data studenta.

TC-06**Výsledek: FAIL****Očekáváno:** HTTP 404 Not Found.**Skutečnost:** HTTP 500 Internal Server Error.

API vrátilo chybu serveru místo informace o neexistujícím studentovi.

TC-07**Výsledek: FAIL****Očekáváno:** HTTP 401 Unauthorized**Skutečnost:** HTTP 403 Forbidden

API vrátilo HTTP 403 Forbidden místo očekávaného HTTP 401 Unauthorized.

TC-08**Výsledek: PASS****Očekáváno:** HTTP 422 Unprocessable Entity**Skutečnost:** HTTP 422 Unprocessable Entity

API vrátilo validační chybu.

TC-09**Výsledek: PASS****Očekáváno:** HTTP 200 OK**Skutečnost:** HTTP 200 OK

Student byl úspěšně aktualizován – změny byly ověřeny SQL dotazem.

TC-10**Výsledek: PASS****Očekáváno:** HTTP 404 Not Found**Skutečnost:** HTTP 404 Not Found

API správně informovalo, že nelze aktualizovat neexistujícího studenta.

TC-11**Výsledek: FAIL****Očekáváno:** HTTP 401 Unauthorized**Skutečnost:** HTTP 403 Forbidden

API vrátilo HTTP 403 Forbidden místo očekávaného HTTP 401 Unauthorized.

TC-12**Výsledek: PASS****Očekáváno:** HTTP 422 Unprocessable Entity**Skutečnost:** HTTP 422 Unprocessable Entity

TC-13

(získané ID = 279)

Výsledek: FAIL

Očekáváno: HTTP 200 OK

Student je odstraněn z databáze.

Skutečnost: HTTP 200 OK

API vrátilo: Student deleted successfully

SQL ověřením bylo zjištěno, že student v databázi stále existuje.

TC-14

Výsledek: FAIL

Očekáváno: HTTP 404 Not Found

Informace, že student neexistuje.

Skutečnost: HTTP 500 Internal Server Error

API vrátilo chybu serveru místo informace o neexistujícím studentovi.

TC-15

Výsledek: FAIL

Očekáváno: HTTP 401 Unauthorized.

Skutečnost: HTTP 403 Forbidden.

API vrátilo HTTP 403 Forbidden místo očekávaného HTTP 401 Unauthorized.

TC-16

Výsledek: PASS

Očekáváno: HTTP 422 Unprocessable Entity

Skutečnost: HTTP 422 Unprocessable Entity

API vrátilo validační chybu.

TC-17

Výsledek: PASS

POST vrátilo HTTP 201 Created a student byl vytvořen. Z odpovědi POST požadavku bylo získáno ID studenta. Následný GET vrátil HTTP 200 OK a data odpovídala údajům použitým při vytvoření studenta.

BUG REPORT

Na základě provedených scénářů jsem objevil(a) uvedené chyby aplikace.

bug	probability	severity	test case number	mitigace
API vrací HTTP 403 místo HTTP 401 při chybějícím Bearer tokenu.	High	Medium	TC-03 TC-07 TC-11 TC-15	Vracet HTTP 401 Unauthorized podle dokumentace.
API vrací HTTP 500 místo HTTP 404 při požadavku na neexistujícího studenta.	High	High	TC-06	Vracet HTTP 404 Not Found podle dokumentace.
API vrací HTTP 200 OK a hlásí úspěšné smazání studenta, ale záznam zůstává uložen v databázi.	High	High	TC-13	Po úspěšném DELETE odstranit záznam z databáze nebo vracet odpovídající chybový stav.
API vrací HTTP 500 místo očekávaného HTTP 404 při pokusu smazat neexistujícího studenta.	High	High	TC-14	Při nenalezení studenta vracet HTTP 404 Not Found místo 500 Internal Server Error.